

JOINS AND ITS TYPES

1. Create table and insert values

Firstly, Created the table Customers and Orders as we are going to perform Join operations

```
SQLQuery1.sql ...U\Krishy (51))*  SQLQuery8.sql...Krishy (53))*  -> X
1  create database ECOM_SITE
2
3  use ECOM_SITE
4  --Customers Table
5  CREATE TABLE Customers (
6      CustID INT PRIMARY KEY,
7      Name VARCHAR(20) NOT NULL,
8      Country VARCHAR(30)
9  );
10
11  --Orders Table
12  CREATE TABLE Orders (
13      OrderID INT PRIMARY KEY,
14      CustID INT NOT NULL,
15      Product VARCHAR(50),
16      Price DECIMAL(10,2),
17  );
18
```

```
19  -- Inserting values into Customers table
20  INSERT INTO Customers (CustID, Name, Country) VALUES
21      (1, 'Tony', 'USA'),
22      (2, 'Thor', 'UK'),
23      (3, 'Steve', 'Canada'),
24      (4, 'Natasha', 'Australia');
25
26  -- Inserting values into Orders table
27  INSERT INTO Orders (OrderID, CustID, Product, Price) VALUES
28      (101, 1, 'Laptop', 1200.00),
29      (102, 2, 'Phone', 800.00),
30      (103, 2, 'Tablet', 600.00),
31      (104, 3, 'Monitor', 300.00);
32
33
```

(4 rows affected)

(4 rows affected)

Completion time: 2025-08-11T23:55:39.2970626+05:30

Types of Joins.

There are 6 types are as follows...

- Inner join
- Outer join
- Right join
- Left join
- Cross join
- Self join

Now we will see one by one...

Inner join

Returns rows that have matching values in both tables.

The screenshot shows a SQL query window with the following code:

```
3 SELECT * FROM Customers;  
4 SELECT * FROM Orders;  
5  
6  
7 --INNER JOIN - Customers who placed orders  
8 SELECT c.CustID, c.Name, c.Country, o.Product, o.Price  
9 FROM Customers c  
10 INNER JOIN Orders o ON c.CustID = o.CustID;  
11
```

The query results are displayed in three tables:

	CustID	Name	Country
1	1	Tony	USA
2	2	Thor	UK
3	3	Steve	Canada
4	4	Natasha	Australia

	OrderID	CustID	Product	Price
1	101	1	Laptop	1200.00
2	102	2	Phone	800.00
3	103	2	Tablet	600.00
4	104	3	Monitor	300.00

	CustID	Name	Country	Product	Price
1	1	Tony	USA	Laptop	1200.00
2	2	Thor	UK	Phone	800.00
3	2	Thor	UK	Tablet	600.00
4	3	Steve	Canada	Monitor	300.00

The last table is the result of Inner join.

Left join

Returns all rows from the left table, and matched rows from the right. Unmatched rows get NULLs.

The screenshot shows a SQL query window with the following code:

```
2 SELECT * FROM Customers;
3
4 SELECT * FROM Orders;
5
6 --LEFT JOIN - List all customers, including who haven't ordered placed any orders.
7 SELECT c.CustID, c.Name, c.Country, o.Product, o.Price
8 FROM Customers c
9 LEFT JOIN Orders o ON c.CustID = o.CustID;
```

The status bar indicates "No issues found". The Results pane shows three tables:

	CustID	Name	Country
1	1	Tony	USA
2	2	Thor	UK
3	3	Steve	Canada
4	4	Natasha	Australia

	OrderID	CustID	Product	Price
1	101	1	Laptop	1200.00
2	102	2	Phone	800.00
3	103	2	Tablet	600.00
4	104	3	Monitor	300.00

	CustID	Name	Country	Product	Price
1	1	Tony	USA	Laptop	1200.00
2	2	Thor	UK	Phone	800.00
3	2	Thor	UK	Tablet	600.00
4	3	Steve	Canada	Monitor	300.00
5	4	Nat...	Austr...	NULL	NULL

The status bar at the bottom indicates "Query executed successfully." and "localhost (16.0 RTM) | SIVU\Krishy (52) | ECOM_SITE | C".

The last table is the result of left join.

Right join

Returns all rows from the right table, and matched rows from the left. Unmatched rows get NULLs.

The screenshot shows a SQL query window with the following code:

```
1 use ECOM_SITE
2
3 SELECT * FROM Customers;
4 SELECT * FROM Orders;
5
6 --RIGHT JOIN - all orders including the orders has no customer details
7 SELECT c.CustID, c.Name, c.Country, o.Product, o.Price
8 FROM Customers c
9 RIGHT JOIN Orders o ON c.CustID = o.CustID;
```

The status bar indicates "No issues found". The Results pane shows three tables:

	CustID	Name	Country
1	1	Tony	USA
2	2	Thor	UK
3	3	Steve	Canada
4	4	Natasha	Australia

	OrderID	CustID	Product	Price
1	101	1	Laptop	1200.00
2	102	2	Phone	800.00
3	103	2	Tablet	600.00
4	104	3	Monitor	300.00

	CustID	Name	Country	Product	Price
1	1	Tony	USA	Laptop	1200.00
2	2	Thor	UK	Phone	800.00
3	2	Thor	UK	Tablet	600.00
4	3	Steve	Canada	Monitor	300.00

The last table is the result of right join.

Full outer join

Returns all rows from both tables. Unmatched rows get NULLs on the side that's missing.

```
2
3  SELECT * FROM Customers;
4  SELECT * FROM Orders;
5
6  --FULL OUTER JOIN - All customers & all orders
7  SELECT c.CustID, c.Name, c.Country, o.Product, o.Price
8  FROM Customers c
9  FULL OUTER JOIN Orders o ON c.CustID = o.CustID;
10
```

0% No issues found

Results Messages

CustID	Name	Country
1	Tony	USA
2	Thor	UK
3	Steve	Canada
4	Natasha	Australia

OrderID	CustID	Product	Price
101	1	Laptop	1200.00
102	2	Phone	800.00
103	2	Tablet	600.00
104	3	Monitor	300.00

CustID	Name	Country	Product	Price
1	Tony	USA	Laptop	1200.00
2	Thor	UK	Phone	800.00
2	Thor	UK	Tablet	600.00
3	Steve	Canada	Monitor	300.00
4	Nat...	Austr...	NULL	NULL

Cross join

- Returns the Cartesian product of two tables. That means every row from the first table is paired with every row from the second table, regardless of any matching keys.
- For instance – Table A has m rows and Table B has n rows.
- The product of m and n will be the output of the Cross join more like a Matrix Multiplication.
- Table A has 3 rows and Table B has 3 rows – the output has 9 rows.

SQLQuery9.sql... \Krishy (52))* SQLQuery1.sql ...U\Krishy (51))*

```

31
32
33 SELECT * FROM Customers;
34 SELECT * FROM Orders;
35
36 --CROSS JOIN - All possible combinations
37 SELECT c.Name, o.Product
38 FROM Customers c
39 CROSS JOIN Orders o;

```

100 % No issues found

Results Messages

	CustID	Name	Country
1	1	Tony	USA
2	2	Thor	UK
3	3	Steve	Canada
4	4	Natasha	Australia

	OrderID	CustID	Product	Price
1	101	1	Laptop	1200.00
2	102	2	Phone	800.00
3	103	2	Tablet	600.00
4	104	3	Monitor	300.00

```

35
36 --CROSS JOIN - All possible combinations
37 SELECT c.Name, o.Product
38 FROM Customers c
39 CROSS JOIN Orders o;

```

00 % No issues found

Results Messages

	Name	Product
1	Tony	Laptop
2	Thor	Laptop
3	Steve	Laptop
4	Natasha	Laptop
5	Tony	Phone
6	Thor	Phone
7	Steve	Phone
8	Natasha	Phone
9	Tony	Tablet
10	Thor	Tablet
11	Steve	Tablet
12	Natasha	Tablet
13	Tony	Monitor
14	Thor	Monitor
15	Steve	Monitor
16	Natasha	Monitor

This is the output of cross join.

Self join

A table joins with itself to compare rows.

The screenshot shows a SQL query window with the following code:

```
40 SELECT * FROM Customers;
41
42 --SELF JOIN - Customers from same country
43 SELECT
44     c1.Name AS Customer1,
45     c2.Name AS Customer2,
46     c1.Country FROM Customers c1
47 JOIN Customers c2
48     ON c1.Country = c2.Country
49     AND c1.CustID <> c2.CustID
50 ORDER BY c1.Country, c1.Name;
51
```

Below the query window, the 'Results' tab is active, displaying two tables. The first table shows the original data, and the second table shows the results of the self-join query.

	CustID	Name	Country
1	1	Tony	USA
2	2	Thor	UK
3	3	Steve	Canada
4	4	Natasha	Australia
5	5	Eva	Australia

	Customer1	Customer2	Country
1	Eva	Natasha	Australia
2	Natasha	Eva	Australia

Thank you!